Jenkins & Playwright — Beginner's Guide

> **Audience**: developers and technical people who are new to Jenkins and Playwright. You should be comfortable with Docker basics (images, containers, `docker-compose up`) but no prior Jenkins or E2E testing experience is assumed.
>
> **What this guide covers**: what Playwright and Jenkins are, how they're set up in HelpDesk Pro, where every environment variable lives, and step-by-step what happens when you trigger a test run — both locally and on OpenShift.
>
> **For Helm template details**: this guide focuses on *concepts and flows*. For the line-by-line YAML template walkthroughs, see the OpenShift & Helm Deep-Dive Guide.

Table of Contents

- Jenkins & Playwright — Beginner's Guide
  - Table of Contents
  - Part 1 — What Are These Tools?
    - 1.1 What is Playwright?
    - 1.2 What is Jenkins?
    - 1.3 How They Work Together
    - 1.4 Key Terms Glossary
  - Part 2 — Playwright in This Project
    - 2.1 The e2e/ Directory
    - 2.2 Test Configuration
    - 2.3 The Page Object Pattern
    - 2.4 Test Data and Cleanup
    - 2.5 How a Test Run Works (Step by Step)
  - Part 3 — Jenkins in This Project
    - 3.1 The Custom Jenkins Docker Image
    - 3.2 Jenkins Plugins
    - 3.3 Configuration as Code (JCasC)
      - Section 1: OIDC Authentication (Login via Keycloak)
      - Section 2: Role-Based Access Control
      - Section 3: Stored Credentials
      - Section 4: Job Definition (Job DSL)
    - 3.4 The "Run-Playwright-Tests" Job
      - Step 1: Parameters (What you can configure before clicking "Build")
      - Step 2: SCM — Clone the Code
      - Step 3: Credentials Binding — Inject Secrets Securely
      - Step 4: Build Step — Run the Tests
      - Step 5: Post-Build — Publish Results
  - Part 4 — Where Do the Environment Variables Live?
    - 4.1 The Variable Story
      - Scene 1 — Image Build Time
      - Scene 2 — Container Startup
      - Scene 3 — The Two-URL Keycloak Problem
      - Scene 4 — JCasC Configures Jenkins
      - Scene 5 — You Trigger a Build
      - Scene 6 — The Tests Run
      - The Full Chain at a Glance
    - 4.2 Complete Variable Map
      - Jenkins Container Variables
      - Playwright Test Variables (inside the build step)
      - Firefox-Specific Variables (set by `ci-entrypoint.sh`)
  - Part 5 — What Happens When You Run It
    - 5.1 Locally (docker-compose)

        - Starting the Stack
        - Jenkins Startup Sequence
        - Why Port 9090?
    - 5.2 On OpenShift
        - Deployment
        - Jenkins Startup on OpenShift
        - Key Differences from Local
- Part 6 — Demo Walkthrough Script
    - Prerequisites
    - The Demo (Step by Step)
        - Step 1 — Show the Application
        - Step 2 — Open Jenkins
        - Step 3 — Explore the Dashboard
        - Step 4 — Trigger a Build
        - Step 5 — Watch the Build
        - Step 6 — View the Report
        - Step 7 — (Optional) Show the Shortcut: Running from Command Line
- Part 7 — How It All Connects
    - Project Files Quick Reference
- Part 8 — Going Deeper
    - In This Project
    - Official Documentation
- Part 9 — Three Alternative Ways to Run Tests with Jenkins
    - 9.1 Jenkinsfile / Pipeline-as-Code
    - 9.2 Docker / Kubernetes Container Agent
    - 9.3 Webhook-Triggered Multibranch Pipeline
- Part 10 — Selenium: An Alternative UI Test Tool
    - 10.1 What is Selenium?
    - 10.2 How Selenium Works in Practice
    - 10.3 Integrating Selenium with Jenkins
    - 10.4 Three Selenium Architectures for Azure DevOps + OpenShift
        - Architecture 1: Selenium on the Jenkins Agent
        - Architecture 2: Selenium Grid with Docker Containers on OpenShift
        - Architecture 3: Selenium Grid 4 Native Kubernetes on OpenShift (Elastic)

Part 1 — What Are These Tools?

1.1 What is Playwright?

Playwright is a tool that tests your application the way a real person would use it.

When you write "unit tests", you're testing small pieces of code in isolation — a single function, a single component. That's great for catching bugs in logic, but it can't tell you if the login page *actually works* when a user types their password and clicks "Sign In".

That's where **end-to-end (E2E) testing** comes in. E2E tests launch a real web browser, navigate to your application, click buttons, fill in forms, and verify that the right things appear on screen. If a user can do it, an E2E test can automate it.

Playwright is Microsoft's open-source E2E testing framework. It:

- **Controls real browsers** — Chromium (Chrome), Firefox, and WebKit (Safari)
- **Runs "headless"** — the browser runs without a visible window, which is perfect for automated testing on a server (like Jenkins). You can also run it "headed" (visible window) for debugging.
- **Handles modern web apps** — it waits for elements to appear, handles single-page app navigation, and deals with authentication flows automatically
- **Generates reports** — HTML reports with screenshots, traces, and detailed failure information

> **In this project**: Playwright tests live in the `e2e/` folder. They test the HelpDesk Pro application by logging in as different users (employee, admin), submitting tickets, checking the admin dashboard, and verifying that everything works end to end.

1.2 What is Jenkins?

Jenkins is a server that runs your tests (and other tasks) automatically.

You *could* run Playwright tests manually from your terminal every time you want to check if the app works. But that's tedious. Jenkins is a **CI/CD server** (Continuous Integration / Continuous Delivery) that:

- **Provides a web dashboard** — you click a button to trigger a test run, and watch the results in your browser
- **Runs jobs automatically** — you can schedule tests, or trigger them when code is pushed to Git
- **Keeps history** — every test run is recorded with logs, reports, and artifacts
- **Manages credentials securely** — passwords and secrets are stored encrypted, not in plain text
- **Controls access** — different users can have different permissions (admin, tester, read-only)

Think of Jenkins as a **robot assistant** that sits on a server, waiting for you to say "run the tests". When you do, it:

1. Downloads the latest code from Git
2. Runs the Playwright tests
3. Collects the results
4. Shows you a nice HTML report

> **In this project**: Jenkins is deployed alongside HelpDesk Pro (both locally and on OpenShift). It comes pre-configured with a "Run-Playwright-Tests" job — no manual setup needed.

1.3 How They Work Together

Here's the relationship in one picture:

```
┌──────────────────────────────────────────────────────────┐
│                        JENKINS                           │
│                   (the orchestrator)                     │
│                                                          │
│  1. You click "Build" in the Jenkins web UI              │
│  2. Jenkins clones the Git repository                    │
│  3. Jenkins runs the Playwright test scripts             │
│                          │                               │
│                          ▼                               │
│   ┌──────────────────────────────────────────────┐       │
│   │              PLAYWRIGHT                      │       │
│   │          (the test runner)                   │       │
│   │                                              │       │
│   │  4. Launches a headless Chromium browser     │       │
│   │  5. Navigates to HelpDesk Pro                │       │
│   │  6. Logs in via Keycloak                     │       │
│   │  7. Submits tickets, checks dashboard, etc.  │       │
│   │  8. Reports pass/fail for each test          │       │
│   └──────────────────────────────────────────────┘       │
│                          │                               │
│  9. Jenkins collects the HTML report                     │
│ 10. You view results in the Jenkins UI                   │
└──────────────────────────────────────────────────────────┘
```

Jenkins is the orchestrator — it triggers, manages, and reports. **Playwright is the test runner** — it does the actual browser automation. Jenkins doesn't know *how* to test a web app. Playwright doesn't know *when* to run or where to put the results. Together, they form a complete automated testing pipeline.

1.4 Key Terms Glossary

You'll encounter these terms throughout this guide and in the project's configuration files. Each one is explained in detail later, but here's a quick reference:

Term	What It Means

Term	What It Means
E2E (End-to-End)	Testing the full application flow as a user would experience it — from the browser through the backend to the database and back.
Headless Browser	A browser running without a visible window. Used for automated testing on servers where there's no screen.
CI/CD	**Continuous Integration** (automatically test code changes) / **Continuous Delivery** (automatically deploy tested code). Jenkins handles the CI part here.
SCM (Source Code Management)	A system that tracks code changes — in practice, this means **Git**. When Jenkins says "SCM", it means "pull the code from the Git repository".
DSL (Domain-Specific Language)	A mini-language designed for one purpose. Jenkins uses **Job DSL** — a Groovy-based language for defining jobs in code instead of clicking through the UI.
JCasC (Jenkins Configuration as Code)	A Jenkins plugin that lets you define *all* Jenkins settings (security, plugins, jobs, credentials) in a YAML file. Instead of configuring Jenkins through its web UI, you write a YAML file and Jenkins configures itself on startup.
OIDC (OpenID Connect)	A login protocol. Instead of Jenkins managing its own usernames and passwords, it delegates authentication to Keycloak. When you click "Log in" on Jenkins, you're redirected to Keycloak's login page.
PVC (PersistentVolumeClaim)	Kubernetes storage that survives Pod restarts. Jenkins uses a PVC to keep its job history and build data even if the Jenkins container is restarted or moved.
Page Object Model	A test design pattern where each page of the app gets its own class with methods like `login()`, `submitTicket()`, etc. Keeps tests clean and reusable.
Job (Jenkins)	A configured task that Jenkins can run. Our project has one job: "Run-Playwright-Tests".
Build (Jenkins)	A single execution of a job. Each time you click "Build", Jenkins creates a new build with a number (#1, #2, #3...).

Part 2 — Playwright in This Project

2.1 The e2e/ Directory

All Playwright tests and configuration live in the `e2e/` folder at the project root. Here's what each part does:

```
e2e/
├── playwright.config.ts       ← Main configuration: which browsers, which tests, what settings
├── global-setup.ts            ← Runs BEFORE all tests (cleans up old test data)
├── global-teardown.ts         ← Runs AFTER all tests (cleans up again)
├── cleanup.ts                 ← Shared cleanup logic (deletes test tickets from the database)
├── package.json               ← npm dependencies (Playwright + TypeScript)
│
├── fixtures/
│   └── test-data.ts           ← Test data: ticket titles, user credentials, expected values
│
├── pages/                     ← Page Objects (one class per app page)
│   ├── login.page.ts          ← Keycloak login page interactions
│   ├── submit-ticket.page.ts  ← Employee ticket submission form
│   ├── admin-dashboard.page.ts← Admin dashboard table, filters, stats
│   └── navigation.page.ts    ← Shared navigation (sidebar, menus)
│
├── tests/                     ← Actual test files, organized by user role
│   ├── auth.setup.ts          ← Authentication setup (logs in and saves session)
│   ├── employee/              ← Tests that run as an employee user
│   │   ├── submit-ticket-form.spec.ts
│   │   ├── submit-ticket-validation.spec.ts
│   │   └── my-tickets.spec.ts
```

```
│   ├── admin/                  ← Tests that run as an admin user
│   │   ├── tickets-table.spec.ts
│   │   ├── dashboard-stats.spec.ts
│   │   ├── filter-sort.spec.ts
│   │   ├── assign-ticket.spec.ts
│   │   └── update-status.spec.ts
│   ├── auth/                   ← Login/logout tests
│   │   ├── login.employee.spec.ts
│   │   ├── login.admin.spec.ts
│   │   ├── logout.employee.spec.ts
│   │   └── logout.admin.spec.ts
│   └── navigation/             ← Navigation tests
│
└── scripts/                    ← Shell scripts for CI automation
    ├── ci-entrypoint.sh        ← Main entry point for Jenkins (orchestrates everything)
    ├── install-deps.sh         ← Installs npm packages + Playwright browsers
    └── run-tests.sh            ← Runs Playwright with the right flags
```

2.2 Test Configuration

The file `e2e/playwright.config.ts` is where Playwright learns *what* to test and *how*. The most important concept here is **projects**.

A **project** is a combination of **browser** + **user role**. For example:

Project Name	Browser	User Role	What It Tests
`setup-chromium`	Chrome	—	Logs in as employee + admin, saves sessions
`setup-firefox`	Firefox	—	Same, but in Firefox
`employee-chromium`	Chrome	employee1	Employee tests (submit tickets, view my tickets)
`employee-firefox`	Firefox	employee1	Same employee tests, in Firefox
`admin-chromium`	Chrome	admin1	Admin tests (dashboard, assign, filter, stats)
`admin-firefox`	Firefox	admin1	Same admin tests, in Firefox

The **setup projects** run first (they log in and save the browser session to a file). Then the **test projects** reuse that saved session — so each test doesn't need to log in again.

Key settings from the config:

```
fullyParallel: false,        // Tests run one at a time (not in parallel)
retries: process.env.CI ? 1 : 0,  // On CI (Jenkins), retry failed tests once
workers: process.env.CI ? 1 : undefined, // On CI, use 1 worker (saves memory)
reporter: process.env.CI ? 'html' : 'list', // On CI, generate HTML report
```

The `baseURL` (which app URL to test) comes from the `BASE_URL` environment variable, defaulting to `http://localhost:4200`.

> **How `page` and Page Objects actually work**
>
> Every test receives a `page` object — a programmatic handle to a **live, headless browser tab**. This is the single object through which all browser interactions happen.
>
> When a test calls `page.goto(BASE_URL)`, Playwright genuinely opens a browser, sends an HTTP request to that URL, and loads whatever HTML/JS the server returns — exactly the same as a human visiting the URL. Angular is just a web server from Playwright's perspective.
>
> When a test calls `page.locator('#some-id')`, Playwright asks the live browser: **"find the element with this CSS selector in whatever HTML you currently have loaded."** The selector is evaluated against the real DOM at the moment the call executes — not against a snapshot or a mirror. Nothing is pre-loaded or cached.

> **Page Objects contain no HTML or DOM.** A class like `LoginPage` is just a named collection of selector strings (`'#username'`, `'#password'`) and wrapper methods (`login()`). When you write `this.usernameInput = page.locator('#username')`, you're storing an address — a promise to ask the live browser for that element when you actually need it.
>
> The practical consequence: if a developer changes a button's ID in the HTML, the selector string in the Page Object no longer matches anything in the live browser, and the test fails. That's intentional — the test is telling you the UI changed.
>
> If you only need to interact with an element once, you don't need a Page Object at all. You can call `page.locator('#some-id')` directly in the test. Page Objects are purely a code-organisation pattern for selectors you'll reuse across many tests.

2.3 The Page Object Pattern

Instead of writing raw browser commands in every test, the project uses the **Page Object Model**. Each page of the app has a corresponding class in `e2e/pages/` that wraps the browser interactions.

For example, `e2e/pages/login.page.ts`:

```
export class LoginPage {
  constructor(page: Page) {
    this.usernameInput = page.locator('#username');
    this.passwordInput = page.locator('#password');
    this.submitButton = page.locator('#kc-login');
  }

  async login(username: string, password: string): Promise<void> {
    await this.usernameInput.fill(username);
    await this.passwordInput.fill(password);
    await this.submitButton.click();
  }
}
```

> **How Playwright connects to the app**: Playwright opens a real browser and navigates to `BASE_URL` (e.g. `http://localhost:4200`). The Angular app, seeing no active session, immediately **redirects the browser to Keycloak's login page**. `LoginPage` targets the HTML elements on *that Keycloak page* — the `#username`, `#password`, and `#kc-login` IDs belong to Keycloak's own login form, not to Angular. So `login.page.ts` is really a "Keycloak login page" object. The `BASE_URL` environment variable is the single wire connecting Playwright to whichever environment is under test.

Then in a test, instead of repeating those three steps every time, you just write:

```
const loginPage = new LoginPage(page);
await loginPage.login('employee1', 'password123');
```

Why this matters: if the login page HTML changes (say, the button ID changes from `#kc-login` to `#login-btn`), you only fix it in one place (the Page Object), not in every test.

2.4 Test Data and Cleanup

Test data is defined in `e2e/fixtures/test-data.ts`. All test tickets have titles starting with `E2E` — this prefix is important because the cleanup logic uses it.

Cleanup (`e2e/cleanup.ts`) runs before *and* after the test suite. It:

1. Gets an admin token from Keycloak
2. Fetches all tickets from the API
3. Deletes any ticket whose title starts with `E2E`

This ensures tests start with a clean slate and don't leave junk data behind.

2.5 How a Test Run Works (Step by Step)

When Playwright tests execute (whether from your terminal or Jenkins), here's the exact sequence:

```
1. GLOBAL SETUP (global-setup.ts)
   └─ Calls cleanup.ts → deletes any leftover "E2E *" tickets from previous runs

2. AUTH SETUP (tests/auth.setup.ts)
   ├─ Launches a browser
   ├─ Navigates to the app → redirected to Keycloak login
   ├─ Logs in as employee1 → saves browser session to .auth/employee-chromium.json
   └─ Logs in as admin1 → saves browser session to .auth/admin-chromium.json

3. TEST EXECUTION (runs each project in order)
   ├─ employee-chromium: loads saved employee session → runs employee/*.spec.ts
   ├─ employee-firefox: loads saved employee session → runs employee/*.spec.ts (Firefox)
   ├─ admin-chromium: loads saved admin session → runs admin/*.spec.ts
   └─ admin-firefox: loads saved admin session → runs admin/*.spec.ts (Firefox)

4. GLOBAL TEARDOWN (global-teardown.ts)
   └─ Calls cleanup.ts → deletes all "E2E *" tickets created during tests

5. REPORTING
   ├─ HTML report generated in e2e/playwright-report/
   ├─ Screenshots saved for failed tests in e2e/test-results/
   └─ Traces saved for retried tests (detailed step-by-step recording)
```

Part 3 — Jenkins in This Project

3.1 The Custom Jenkins Docker Image

Jenkins doesn't come with Node.js or Playwright browsers out of the box. Since our tests need both, the project builds a **custom Jenkins image** (defined in `jenkins/Dockerfile`) that includes everything:

```
Official Jenkins LTS (Java 17)
    │
    ├── + Node.js 20          ← Needed to run npm and Playwright
    ├── + Chromium browser     ← Pre-installed so tests don't download it every time
    ├── + Firefox browser      ← Same — pre-installed for speed
    ├── + Jenkins plugins      ← From jenkins/plugins.txt (see section 3.2)
    └── + Writable directories ← For OpenShift compatibility (runs as random non-root user)
```

Why pre-install browsers? Playwright browsers are ~400MB. If Jenkins had to download them every time a test runs, each build would waste minutes just downloading. By baking them into the Docker image, tests start immediately.

Key lines in the Dockerfile explained:

```
# Pre-install browsers at image build time
ENV PLAYWRIGHT_BROWSERS_PATH=/ms-playwright
COPY e2e/package.json e2e/package-lock.json* /tmp/e2e/
RUN cd /tmp/e2e && npm ci && npx playwright install --with-deps chromium firefox
```

This copies the `e2e/package.json` into the image, installs dependencies, and downloads Chromium + Firefox to `/ms-playwright`. The `--with-deps` flag also installs system libraries the browsers need (fonts, graphics libraries, etc.).

```
# Disable the setup wizard — JCasC handles everything
ENV JAVA_OPTS="... -Djenkins.install.runSetupWizard=false ..."
```

Normally, when you start Jenkins for the first time, it shows a setup wizard asking you to create an admin user and pick plugins. We skip this because JCasC configures everything automatically.

```
# Tell Jenkins where to find the JCasC config file
ENV CASC_JENKINS_CONFIG="/var/jenkins_home/casc_configs/"
```

This points Jenkins to the directory where the Configuration as Code YAML file will be mounted.

3.2 Jenkins Plugins

The file `jenkins/plugins.txt` lists the plugins installed in the custom image:

Plugin	What It Does
`oic-auth`	**OpenID Connect authentication** — lets Jenkins delegate login to Keycloak instead of managing its own passwords
`role-strategy`	**Role-based access control** — lets us map Keycloak roles (helpdesk-admin, helpdesk-tester) to Jenkins permissions
`configuration-as-code`	**JCasC** — configure Jenkins entirely from a YAML file (no manual UI setup)
`git`	**Git integration** — lets Jenkins clone repositories (this is the SCM plugin)
`workflow-aggregator`	**Pipeline support** — the core pipeline engine (even though we use a freestyle job, this is a common dependency)
`htmlpublisher`	**HTML reports** — publishes the Playwright HTML report so you can view it in Jenkins
`nodejs`	**Node.js tool** — registers Node.js as a build tool in Jenkins
`job-dsl`	**Job DSL** — lets us define jobs in Groovy code (inside the JCasC YAML) instead of creating them manually through the UI
`credentials`	**Credentials store** — securely stores passwords, tokens, secrets
`credentials-binding`	**Credentials binding** — injects stored credentials into build steps as environment variables

3.3 Configuration as Code (JCasC)

This is the most important piece to understand. Instead of clicking through Jenkins' web UI to set up security, credentials, and jobs, everything is declared in a single YAML file: `jenkins/casc/jenkins.yaml`.

When Jenkins starts, it reads this file and configures itself. **No manual setup, no clicking, completely reproducible.**

The JCasC file has four main sections:

Section 1: OIDC Authentication (Login via Keycloak)

```
securityRealm:
  oic:
    clientId: "helpdesk-jenkins"
    clientSecret: "${JENKINS_OIDC_CLIENT_SECRET}"
    # ... Keycloak endpoint URLs ...
```

What this does: When someone visits Jenkins and isn't logged in, Jenkins redirects them to Keycloak's login page. After the user logs in with their Keycloak credentials, Keycloak sends Jenkins a token that says "this is user admin1, they have roles helpdesk-admin and employee". Jenkins uses this to determine who they are and what they can do.

The `helpdesk-jenkins` client: This is a Keycloak client specifically for Jenkins (defined in the Keycloak realm). It's different from `helpdesk-frontend` (for the Angular app) and `helpdesk-backend` (for the .NET API). Each application that authenticates with Keycloak gets its own client.

The client secret (`${JENKINS_OIDC_CLIENT_SECRET}`): This is a shared password between Jenkins and Keycloak that proves Jenkins is who it claims to be. The `${...}` syntax means "read this from an environment variable" — so the secret isn't hardcoded in the YAML file.

Section 2: Role-Based Access Control

```
authorizationStrategy:
  roleBased:
    roles:
      global:
        - name: "admin"
          permissions: ["Overall/Administer"]
          entries:
            - group: "helpdesk-admin"
        - name: "tester"
          permissions: ["Job/Build", "Job/Read", ...]
          entries:
            - group: "helpdesk-tester"
        - name: "readonly"
          permissions: ["Overall/Read"]
          entries:
            - group: "authenticated"
```

What this does: Maps Keycloak roles to Jenkins permissions.

If your Keycloak role is...	You get Jenkins role...	Which means you can...
`helpdesk-admin`	admin	Do everything (full admin access)
`helpdesk-tester`	tester	Trigger builds, view results, cancel jobs
(any authenticated user)	readonly	View the dashboard (read-only)

So `admin1` (who has the `helpdesk-admin` role in Keycloak) gets full Jenkins admin access. `tester1` (who has `helpdesk-tester`) can trigger test runs but can't change Jenkins configuration. `employee1` (who has neither) gets read-only access.

Section 3: Stored Credentials

```
credentials:
  system:
    domainCredentials:
      - credentials:
          - string:
              id: "employee-username"
              secret: "${EMPLOYEE_USERNAME:-employee1}"
          - string:
              id: "employee-password"
              secret: "${EMPLOYEE_PASSWORD:-password123}"
          # ... same for admin ...
```

What this does: Stores the test user credentials (employee1/password123, admin1/password123) in Jenkins' encrypted credential store. When the Playwright tests run, they need to log into the HelpDesk app — these credentials tell them which username and password to use.

The `:-` syntax: `${EMPLOYEE_USERNAME:-employee1}` means "use the `EMPLOYEE_USERNAME` environment variable, but if it's not set, default to `employee1`". This makes the config work both locally (where the defaults are fine) and on OpenShift (where you might need different credentials).

Section 4: Job Definition (Job DSL)

```
jobs:
  - script: |
      job('Run-Playwright-Tests') {
        description('Runs Playwright E2E tests against HelpDesk Pro')
        parameters {
          stringParam('BASE_URL', ...)
          choiceParam('BROWSER_PROJECT', ['chromium', 'all', 'firefox', ...])
          stringParam('TEST_RETRIES', '1', ...)
        }
        scm { git { ... } }
        wrappers { credentialsBinding { ... } }
        steps { shell('cd e2e && ./scripts/ci-entrypoint.sh') }
        publishers { publishHtml { ... } }
      }
```

What this does: Automatically creates a Jenkins job called "Run-Playwright-Tests" when Jenkins starts. This is written in **Job DSL** — a Groovy-based language for defining jobs in code. The next section breaks this down in detail.

3.4 The "Run-Playwright-Tests" Job

This is the job you'll interact with. It's auto-created by JCasC (Section 4 above). Here's what each part does:

Step 1: Parameters (What you can configure before clicking "Build")

When you click "Build with Parameters", you see three options:

Parameter	Default	What It Controls
BASE_URL	`http://frontend:8080` (local) or the OpenShift frontend route URL	The URL Playwright will test against. Change this if you want to test a different environment.
BROWSER_PROJECT	`chromium`	Which browser(s) to test. Options: `chromium` (fast, recommended), `firefox`, `all` (both — needs more memory).
TEST_RETRIES	`1`	How many times to retry a failed test before marking it failed. `1` means each test gets two chances.

Step 2: SCM — Clone the Code

```
scm {
  git {
    remote { url('https://github.com/maguiarr/Helpdesk.git') }
    branch('main')
  }
}
```

SCM = Source Code Management, which in practice means **Git**. This tells Jenkins: "Before running the tests, clone the `main` branch of the repository." This ensures Jenkins always tests the latest code, including any changes to the test files themselves.

What credentials does Jenkins use to clone? None — the repository is public on GitHub. Because the URL is plain HTTPS (`https://github.com/...`), Git can clone it without authentication. If the repository were private, you'd add a stored credential reference inside the `git {}` block pointing to a username/token stored in Jenkins' vault (the same vault used in Step 3).

Where do the files go? Jenkins automatically creates and manages a **workspace directory** for each job inside the container:

```
/var/jenkins_home/workspace/Run-Playwright-Tests/
                              │
                              ├─ backend/
                              ├─ frontend/
                              ├─ e2e/                ← what the tests need
                              │   ├─ playwright.config.ts
                              │   ├─ pages/
                              │   ├─ tests/
                              │   └─ scripts/
                              │       └─ ci-entrypoint.sh
                              ├─ helm/
                              └─ ...
```

The entire repository is cloned into this workspace. Jenkins automatically sets the shell's working directory to the workspace root before running any build steps — so the repo root is `$PWD` when Step 4 begins.

Why clone? Jenkins' Docker image has Playwright pre-installed (`/ms-playwright`), but it doesn't include the project's test scripts. The image is built once and shared — the SCM step is what fetches the latest version of `e2e/` for each build.

Step 3: Credentials Binding — Inject Secrets Securely

```
wrappers {
  credentialsBinding {
    string('EMPLOYEE_USERNAME', 'employee-username')
    string('EMPLOYEE_PASSWORD', 'employee-password')
    string('ADMIN_USERNAME', 'admin-username')
    string('ADMIN_PASSWORD', 'admin-password')
  }
}
```

This takes the credentials stored in Section 3 (JCasC) and makes them available as **environment variables** during the build. So when the test script runs, it can read `$EMPLOYEE_USERNAME` and `$EMPLOYEE_PASSWORD` to log into the app.

Why not just hardcode them? Credentials binding means passwords are:

- Encrypted at rest in Jenkins
- Masked in build logs (you'll see `****` instead of the actual password)
- Managed centrally (change in one place, applies to all jobs)

Step 4: Build Step — Run the Tests

```
steps {
  shell('''
    cd e2e
    chmod +x scripts/*.sh
    ./scripts/ci-entrypoint.sh
  ''')
}
```

Jenkins runs this shell script with `$PWD` already set to the workspace root (where the repo was cloned in Step 2). So the three lines do:

1. `cd e2e` — moves into the `e2e/` subfolder of the cloned repo (`/var/jenkins_home/workspace/Run-Playwright-Tests/e2e/`)
2. `chmod +x scripts/*.sh` — makes the shell scripts executable (Git doesn't always preserve file permissions on clone)

3. `./scripts/ci-entrypoint.sh` — runs the entrypoint script, which now finds all the test files at relative paths it already expects (`./pages/`, `./tests/`, etc.) because it's running from inside `e2e/`

What `ci-entrypoint.sh` does (the CI pipeline within the build step):

```
ci-entrypoint.sh
│
├── Step 1: install-deps.sh
│   ├── npm ci (install Node.js packages)
│   └── Check if Playwright browsers exist at /ms-playwright (they do — pre-installed in image)
│
├── Step 2: Health check
│   └── Ping the backend's /healthz endpoint up to 30 times
│       (wait for the app to be ready before running tests)
│
├── Step 3: Export credentials
│   └── Set EMPLOYEE_USERNAME, EMPLOYEE_PASSWORD, etc. from Jenkins env vars
│
└── Step 4: run-tests.sh
    ├── Build the Playwright command based on BROWSER_PROJECT
    ├── Set retries, reporter, workers from env vars
    └── Execute: npx playwright test --project=employee-chromium --project=admin-chromium ...
```

Step 5: Post-Build — Publish Results

```
publishers {
  publishHtml {
    report('e2e/playwright-report') {
      reportName('Playwright Report')
      alwaysLinkToLastBuild(true)
    }
  }
  archiveArtifacts {
    pattern('e2e/test-results/**')
    allowEmpty(true)
  }
}
```

After the tests finish (pass or fail), Jenkins:

- **Publishes the HTML report** — the Playwright HTML report (with test details, screenshots, traces) becomes viewable directly in the Jenkins UI as "Playwright Report"
- **Archives artifacts** — test result files (screenshots, trace files) are saved as downloadable build artifacts

---

Part 4 — Where Do the Environment Variables Live?

The variables can feel overwhelming because they're defined in **five different places** and pass through several handoffs before a Playwright test ever runs. This section tells the full story — *why* the system is designed this way, and *what* each handoff does. The quick-reference tables are in section 4.2 once the story makes sense.

4.1 The Variable Story

Follow Jenkins from first boot to first test run. At each stage, a new group of variables enters the picture.

Scene 1 — Image Build Time

When it happens: `docker build` (locally) or the GitHub Actions image build push (CI/CD). **File in charge**: `jenkins/Dockerfile`

Before any container starts, the Docker image is assembled. The Dockerfile bakes in three variables that are the same in *every* environment — they're structural facts about the image itself:

```
┌──────────────────────────────────────────────────────────────┐
│                      JENKINS IMAGE (built once)              │
│                                                              │
│  PLAYWRIGHT_BROWSERS_PATH = /ms-playwright                   │
│    → tells every subsequent process where Chromium/Firefox live  │
│                                                              │
│  CASC_JENKINS_CONFIG = /var/jenkins_home/casc_configs/       │
│    → tells Jenkins where to look for its YAML configuration  │
│                                                              │
│  JAVA_OPTS = "-Xmx384m ... -DrunSetupWizard=false ..."       │
│    → JVM memory limits + skips the first-time setup wizard   │
└──────────────────────────────────────────────────────────────┘
```

> **Key idea**: These variables never change between environments. They describe how the image works internally, not how it connects to the outside world. You can override them at runtime, but you rarely need to.

Scene 2 — Container Startup

When it happens: `make up` (locally) or `helm install` (OpenShift). **Files in charge**: `docker-compose.yml` / `helm/templates/jenkins-deployment.yaml`

When the container starts, the runtime environment injects the **"address book"** — where is everything else? These are the variables that differ between local and OpenShift, because the network addresses are different.

```
LOCAL (docker-compose.yml injects):
──────────────────────────────────
KEYCLOAK_URL          = http://keycloak:8180
KEYCLOAK_BROWSER_URL  = http://localhost:8180
JENKINS_OIDC_CLIENT_SECRET = jenkins-secret
DEFAULT_BASE_URL      = http://frontend:8080
JENKINS_ADMIN_PASSWORD = admin

OPENSHIFT (jenkins-deployment.yaml injects):
────────────────────────────────────────────
KEYCLOAK_URL      = https://{keycloak-route}
(no KEYCLOAK_BROWSER_URL — see Scene 3)
JENKINS_OIDC_CLIENT_SECRET = from K8s Secret
DEFAULT_BASE_URL  = https://{frontend-route}
JENKINS_ADMIN_PASSWORD = from K8s Secret
HEALTH_CHECK_URL  = http://backend:8080/healthz
```

Notice that locally, secrets are plain text in `docker-compose.yml` (convenient for development). On OpenShift, they come from Kubernetes Secrets (encrypted, managed by the platform).

> **Why is `HEALTH_CHECK_URL` only set on OpenShift?** Locally, `ci-entrypoint.sh` falls back to `$BASE_URL/api/healthz`, which works fine. On OpenShift, `DEFAULT_BASE_URL` is a public HTTPS route, but the healthcheck needs to reach the backend *internally* without going through the public internet — so it uses the internal Kubernetes service name instead.

Scene 3 — The Two-URL Keycloak Problem

> This is the trickiest part of the whole variable setup. Read it carefully — it trips up most people.

When you log into Jenkins, **two completely different network paths** are involved:

1. **Your browser** opens the Jenkins page, Jenkins redirects it to the Keycloak login page. Your browser needs a URL *your laptop* can reach.
2. **Jenkins itself** (running inside Docker) needs to call Keycloak's API to exchange the login token and verify your identity. Jenkins needs a URL *the Docker container* can reach.

Locally, these are completely different addresses:

```
YOUR LAPTOP                          DOCKER NETWORK
────────────                         ───────────────

                                     ┌──────────────────────┐
Your browser ── localhost:8180 ──►   │                      │
                                     │   Keycloak container │
Jenkins JVM  ── keycloak:8180 ──►    │   (internal hostname) │
                                     └──────────────────────┘
```

Your browser has no idea what `keycloak` means as a hostname — that name only exists inside the Docker virtual network. Similarly, `localhost:8180` inside the Jenkins container points to Jenkins itself, not Keycloak.

Solution: two separate variables.

Variable	Used For	Local Value	OpenShift Value
`KEYCLOAK_BROWSER_URL`	Browser redirects (login, logout, issuer discovery)	`http://localhost:8180`	*(not set)*
`KEYCLOAK_URL`	Server-to-server API calls (token exchange, JWKS, userinfo)	`http://keycloak:8180`	`https://{keycloak-route}`

On OpenShift, both Jenkins and your browser use the public HTTPS Route URL — it's accessible from everywhere, so one variable handles both.

```
LOCAL:
  Browser redirects  → KEYCLOAK_BROWSER_URL (http://localhost:8180)
  Token exchange     → KEYCLOAK_URL         (http://keycloak:8180)

OPENSHIFT:
  Browser redirects  → KEYCLOAK_URL         (https://keycloak-route-host)
  Token exchange     → KEYCLOAK_URL         (https://keycloak-route-host)
  (one URL works for both — KEYCLOAK_BROWSER_URL is not set)
```

Scene 4 — JCasC Configures Jenkins

When it happens: ~30 seconds after the container starts, while Jenkins initializes. **File in charge**: `jenkins/casc/jenkins.yaml`

Jenkins reads the YAML file mounted at `CASC_JENKINS_CONFIG` and configures itself. Every `${VAR_NAME}` in the YAML is substituted with the environment variable from Scene 2. Three important things happen here:

① Keycloak login is wired up

The JCasC YAML uses the two Keycloak URL variables precisely:

```
KEYCLOAK_BROWSER_URL → authorizationServerUrl, endSessionUrl, issuer  (browser-facing)
KEYCLOAK_URL         → tokenServerUrl, jwksServerUrl, userInfoServerUrl (server-to-server)
JENKINS_OIDC_CLIENT_SECRET → clientSecret (the shared password with Keycloak)
```

After this, clicking "Log in" on Jenkins redirects to Keycloak, and the token handshake works.

② Test credentials are stored in the Jenkins vault

JCasC reads `EMPLOYEE_USERNAME`, `EMPLOYEE_PASSWORD`, `ADMIN_USERNAME`, `ADMIN_PASSWORD` from the environment (using defaults of `employee1` / `password123` if not set) and stores them **encrypted** inside Jenkins' credential store. From this point on, the plain-text values are gone — Jenkins only exposes them to build steps through a masking mechanism.

```
                    ┌──────────────────────────────┐
Env var             │     JENKINS CREDENTIAL       │
EMPLOYEE_USERNAME   │  id: "employee-username"     │  ← encrypted at rest
EMPLOYEE_PASSWORD   │  id: "employee-password"     │  ← masked as **** in logs
ADMIN_USERNAME      │  id: "admin-username"        │
ADMIN_PASSWORD      │  id: "admin-password"        │
                    └──────────────────────────────┘
```

③ The test job is created

Job DSL (inside the same JCasC YAML) creates the "Run-Playwright-Tests" job. `DEFAULT_BASE_URL` is read here and becomes the **default value** for the job's `BASE_URL` parameter — so when a user opens "Build with Parameters", the right URL is already pre-filled.

Scene 5 — You Trigger a Build

When it happens: You click "Build with Parameters" in the Jenkins UI. **File in charge**: The job definition (inside `jenkins/casc/jenkins.yaml`)

This is the only stage where a *human* injects variables. You can override three things:

```
┌──────────────────────────────────────────────────────────────────┐
│                  "Build with Parameters" dialog                  │
│                                                                  │
│  BASE_URL        [http://frontend:8080          ]  ← pre-filled from │
│                                                       DEFAULT_BASE_URL│
│  BROWSER_PROJECT [chromium ▼]                                    │
│  TEST_RETRIES    [1                             ]                │
└──────────────────────────────────────────────────────────────────┘
```

Want to test a staging environment instead of the local one? Change `BASE_URL`. Want to run only Firefox? Change `BROWSER_PROJECT`. Otherwise, leave the defaults. These parameters only affect the test run — they don't change Jenkins configuration.

Scene 6 — The Tests Run

When it happens: Jenkins executes the shell build step. **File in charge**: `e2e/scripts/ci-entrypoint.sh`

The credentials from Scene 4's vault are now injected into the build environment by the credentials binding wrapper, masked as `****` in the logs:

```
Vault "employee-username" → env var EMPLOYEE_USERNAME  (shown as **** in logs)
Vault "employee-password" → env var EMPLOYEE_PASSWORD  (shown as **** in logs)
Vault "admin-username"    → env var ADMIN_USERNAME     (shown as **** in logs)
Vault "admin-password"    → env var ADMIN_PASSWORD     (shown as **** in logs)
```

Then `ci-entrypoint.sh` takes over and adds the remaining runtime vars:

```
ci-entrypoint.sh also sets:
  TEST_REPORTER   = "list,html"              ← output formats
  TEST_WORKERS    = 1                        ← serialize tests (memory constraint)
  NODE_OPTIONS    = "--max-old-space-size=256" ← cap Node.js heap
  TEST_RETRIES    = from job parameter (Scene 5)
  BROWSER_PROJECT = from job parameter (Scene 5)

  If Firefox: (OpenShift security workarounds)
    MOZ_DISABLE_CONTENT_SANDBOX = 1
```

```
    MOZ_DISABLE_GMP_SANDBOX = 1
    MOZ_DISABLE_SOCKET_PROCESS_SANDBOX = 1
    MOZ_DISABLE_RDD_SANDBOX = 1
    HOME = /tmp/firefox-home     ← writable by any UID
    FONTCONFIG_CACHE = /tmp/firefox-home/.cache/fontconfig
```

At the end, `npx playwright test` runs. Playwright reads `BASE_URL`, `EMPLOYEE_USERNAME`, `EMPLOYEE_PASSWORD`, `ADMIN_USERNAME`, `ADMIN_PASSWORD`, `PLAYWRIGHT_BROWSERS_PATH` — all of which have been passed down through the chain above.

The Full Chain at a Glance

```
Stage 1: docker build       → bakes PLAYWRIGHT_BROWSERS_PATH, CASC_JENKINS_CONFIG, JAVA_OPTS
              │
Stage 2: container starts   → injects KEYCLOAK_URL, KEYCLOAK_BROWSER_URL,
              │                       JENKINS_OIDC_CLIENT_SECRET, DEFAULT_BASE_URL, ...
              │
Stage 3: JCasC reads YAML   → wires Keycloak login (KEYCLOAK_URL + KEYCLOAK_BROWSER_URL)
              │                stores test creds encrypted in vault (EMPLOYEE_*, ADMIN_*)
              │                creates job with DEFAULT_BASE_URL as default parameter
              │
Stage 4: user clicks Build  → injects BASE_URL, BROWSER_PROJECT, TEST_RETRIES
              │
Stage 5: build step runs    → vault creds injected as masked env vars
                              ci-entrypoint.sh adds TEST_WORKERS, TEST_REPORTER,
                              NODE_OPTIONS, Firefox sandbox workarounds
                              └─ Playwright runs with the complete set
```

4.2 Complete Variable Map

Here's every environment variable, what it does, and exactly where it's defined:

Jenkins Container Variables

Variable	Purpose	Local (`docker-compose.yml`)	OpenShift (`jenkins-deployment.yaml`)
`JAVA_OPTS`	JVM memory limits, disable setup wizard, CSP for HTML reports	Set in `Dockerfile`	Set in Deployment `env:`
`CASC_JENKINS_CONFIG`	Path to JCasC config files	Set in `Dockerfile` (`/var/jenkins_home/casc_configs/`)	Set in Deployment `env:`
`KEYCLOAK_URL`	Keycloak URL for **server-to-server** calls (token exchange, JWKS)	`http://keycloak:8180` (internal Docker network)	`https://{keycloak-route-host}` (public URL)
`KEYCLOAK_BROWSER_URL`	Keycloak URL for **browser redirects** (login page)	`http://localhost:8180` (user's browser)	*(not set — same as `KEYCLOAK_URL` since both are public on OpenShift)*
`JENKINS_OIDC_CLIENT_SECRET`	Shared secret between Jenkins and Keycloak	`jenkins-secret` (plain text in compose)	From `jenkins-secret.yaml` (Kubernetes Secret)
`DEFAULT_BASE_URL`	Default app URL for Playwright tests	`http://frontend:8080` (internal)	`https://{frontend-route-host}` (public)

Variable	Purpose	Local (`docker-compose.yml`)	OpenShift (`jenkins-deployment.yaml`)
`HEALTH_CHECK_URL`	Backend health endpoint for pre-test check	*(not set — script defaults to `$BASE_URL/api/healthz`)*	`http://helpdesk-pro-backend:8080/healthz` (internal)
`JENKINS_ADMIN_PASSWORD`	Fallback admin password if Keycloak is unavailable	`admin`	From `jenkins-secret.yaml`
`PLAYWRIGHT_BROWSERS_PATH`	Where Playwright browsers are installed	Set in `Dockerfile` (`/ms-playwright`)	Same (inherited from image)

Local-only variables (needed because Docker networking works differently):

Variable	Purpose	Why It's Needed Locally
`KEYCLOAK_BROWSER_URL`	The Keycloak URL as seen by your browser	Locally, your browser can't reach `keycloak:8180` (that's the Docker internal hostname). It needs `localhost:8180`. On OpenShift, the Route's public URL works for both.

Playwright Test Variables (inside the build step)

Variable	Purpose	Set By
`BASE_URL`	The app URL Playwright navigates to	Job parameter (user sets it, or defaults to `DEFAULT_BASE_URL`)
`EMPLOYEE_USERNAME`	Test user login name	JCasC credentials → credentials binding
`EMPLOYEE_PASSWORD`	Test user login password	JCasC credentials → credentials binding
`ADMIN_USERNAME`	Admin test user login name	JCasC credentials → credentials binding
`ADMIN_PASSWORD`	Admin test user login password	JCasC credentials → credentials binding
`BROWSER_PROJECT`	Which browser projects to run	Job parameter
`TEST_RETRIES`	Number of retry attempts	Job parameter
`TEST_WORKERS`	Number of parallel test workers	Set by `ci-entrypoint.sh` (= `1` to save memory)
`TEST_REPORTER`	Output format	Set by `ci-entrypoint.sh` (= `list,html`)
`CI`	Signals "running in CI mode"	*(set automatically by Jenkins)*
`NODE_OPTIONS`	Node.js memory limit	Set by `ci-entrypoint.sh` (`--max-old-space-size=256`)

Firefox-Specific Variables (set by `ci-entrypoint.sh`)

If Firefox tests run, the CI script sets these to work around OpenShift security restrictions:

Variable	Purpose
`MOZ_DISABLE_CONTENT_SANDBOX`	Disables Firefox content sandbox (OpenShift blocks `clone(CLONE_NEWUSER)`)
`MOZ_DISABLE_GMP_SANDBOX`	Disables GMP plugin sandbox
`MOZ_DISABLE_SOCKET_PROCESS_SANDBOX`	Disables socket process sandbox
`MOZ_DISABLE_RDD_SANDBOX`	Disables RDD sandbox
`HOME`	Set to `/tmp/firefox-home` (writable directory for Firefox profile)
`FONTCONFIG_CACHE`	Writable font cache for headless Firefox

> These Firefox variables are only relevant when running on OpenShift, where containers run as a random non-root user ID with restricted security contexts.

Part 5 — What Happens When You Run It

5.1 Locally (docker-compose)

Here's exactly what happens when you run the system locally:

Starting the Stack

```
make up    # or: docker-compose up -d --build
```

This starts **five services** in dependency order:

```
1. postgres        (port 5432) — Database starts first
      ↓ healthy
2. keycloak        (port 8180) — Identity provider, imports realm config
      ↓ healthy                ↓ healthy
3. backend         (port 8080) 5. jenkins    (port 9090)
      ↓ healthy                   CI server, connects to
4. frontend        (port 4200)    keycloak for login
      Angular app (nginx)
```

> **Note**: Backend → Frontend and Jenkins are **parallel branches** — both depend on Keycloak, but Jenkins does **not** depend on Frontend or Backend. In practice, all five services start roughly together; Docker just waits for each service's healthcheck dependencies before marking it healthy.

Jenkins Startup Sequence

Jenkins is the slowest to start (~2 minutes). Here's what happens inside:

```
1. Jenkins JVM starts
   └─ JAVA_OPTS: -Xmx384m, setup wizard disabled, CSP relaxed for HTML reports

2. Plugins load
   └─ oic-auth, configuration-as-code, job-dsl, htmlpublisher, git, etc.

3. JCasC applies (reads /var/jenkins_home/casc_configs/jenkins.yaml)
   ├─ Configures Keycloak OIDC login
   │   └─ Uses KEYCLOAK_URL (http://keycloak:8180) for token endpoints
   │   └─ Uses KEYCLOAK_BROWSER_URL (http://localhost:8180) for browser redirects
   ├─ Sets up role mapping (admin → helpdesk-admin, tester → helpdesk-tester)
   ├─ Stores test credentials (employee1, admin1)
   └─ Creates "Run-Playwright-Tests" job via Job DSL
       └─ DEFAULT_BASE_URL (http://frontend:8080) becomes the job's default BASE_URL

4. Jenkins is ready
   └─ Web UI available at http://localhost:9090
```

Why Port 9090?

In `docker-compose.yml`, Jenkins maps `9090:8080` — the container's port 8080 (Jenkins' internal port) is exposed as 9090 on your machine. This avoids conflicting with the backend, which also uses port 8080.

5.2 On OpenShift

On OpenShift, the flow is similar but the deployment mechanism is different.

Deployment

```
helm install helpdesk-pro ./helm -f helm/values.openshift.yaml
```

Helm renders all templates using values from `values.yaml` + `values.openshift.yaml`, creating ~22 Kubernetes resources including the Jenkins Deployment, Service, Route, PVC, Secret, and ConfigMap.

Jenkins Startup on OpenShift

```
1. Deployment creates a Pod
   ├─ Mounts PVC (5Gi) at /var/jenkins_home (persistent storage)
   ├─ Mounts ConfigMap (JCasC) at /var/jenkins_home/casc_configs/
   └─ Mounts emptyDir at /dev/shm (64Mi shared memory for Chromium)

2. Environment variables injected from Deployment spec:
   ├─ KEYCLOAK_URL = https://{keycloak-route-host}   (public HTTPS URL)
   ├─ DEFAULT_BASE_URL = https://{frontend-route-host} (public HTTPS URL)
   ├─ HEALTH_CHECK_URL = http://helpdesk-pro-backend:8080/healthz (internal)
   ├─ JENKINS_OIDC_CLIENT_SECRET = from jenkins-secret (Kubernetes Secret)
   └─ JENKINS_ADMIN_PASSWORD = from jenkins-secret

3. JCasC applies (same process as local, but URLs are HTTPS public routes)
   └─ All Keycloak endpoints point to public route (no separate BROWSER_URL needed)

4. Health probes start checking
   ├─ Readiness: GET /login (passes after ~90s)
   └─ Liveness: GET /login (passes after ~120s)

5. Route becomes active
   └─ Jenkins accessible at: https://helpdesk-pro-jenkins-{namespace}.apps.{cluster}/
```

Key Differences from Local

Aspect	Local (docker-compose)	OpenShift
URLs	`http://localhost:PORT`	`https://{auto-generated-route-host}`
Keycloak URLs	Two variables: `KEYCLOAK_URL` (internal) + `KEYCLOAK_BROWSER_URL` (browser)	One variable: `KEYCLOAK_URL` (public HTTPS works for both)
Secrets	Plain text in `docker-compose.yml`	Kubernetes Secrets (encrypted, reference by name)
Storage	Docker volume (`jenkins_home:`)	PVC (Kubernetes-managed persistent storage)
JCasC source	File mount: `./jenkins/casc/jenkins.yaml` mounted into container	ConfigMap: `jenkins-casc-configmap.yaml` template rendered by Helm
Network	Docker bridge network (services reach each other by name)	Kubernetes Services + DNS (same idea, different implementation)
Startup time	~2 minutes	~2 minutes (same, but longer probe delays before OpenShift considers it "ready")

> **For the full Helm template walkthroughs** — what each YAML field does, how the Deployment, Service, Route, PVC, Secret, and ConfigMap templates work — see the OpenShift & Helm Deep-Dive Guide, Section 3.5 (Jenkins).

Part 6 — Demo Walkthrough Script

Use this as a script when demonstrating Jenkins + Playwright to others. Each step includes what to show and what to say.

Prerequisites

- All services running: `make up` (wait ~3 minutes for everything to be healthy)
- Verify services are up: `make logs` (check for errors)

Service URLs:

Service	URL
HelpDesk App	http://localhost:4200
Jenkins	http://localhost:9090
Keycloak Admin	http://localhost:8180

The Demo (Step by Step)

Step 1 — Show the Application

1. Open http://localhost:4200 in your browser
2. You'll be redirected to the Keycloak login page
3. Log in as `employee1` / `password123`
4. You'll land on the ticket submission page

> **Say**: "*This is HelpDesk Pro — a support ticket application. Employees can submit tickets, and admins can manage them. Right now I'm logged in as an employee. Let me submit a ticket so we have some data.*"

5. Submit a ticket with any title and description
6. Point out the "My Tickets" section below

> **Say**: "*Now the question is: how do we know this application actually works correctly? We could click around manually every time we make a change, but that's slow and error-prone. That's where automated testing comes in.*"

Step 2 — Open Jenkins

1. Open http://localhost:9090 in a new tab
2. You'll be redirected to Keycloak's login page (same login page as the app!)

> **Say**: "*Notice we're redirected to Keycloak — the same identity provider the app uses. Jenkins doesn't manage its own user accounts. It delegates authentication to Keycloak via OpenID Connect. This means our Keycloak users and roles work in Jenkins too.*"

3. Log in as `admin1` / `password123`
4. You'll land on the Jenkins dashboard

> **Say**: "*I logged in as admin1, who has the helpdesk-admin role in Keycloak. Jenkins maps that to full admin access. If I had logged in as tester1, I'd only be able to trigger builds, not change settings.*"

Step 3 — Explore the Dashboard

1. Point out the "Run-Playwright-Tests" job in the job list

> **Say**: "*This job was created automatically when Jenkins started — we didn't click anything to set it up. It's defined in code, in a Configuration-as-Code YAML file. If we delete this Jenkins instance and start a new one, the job reappears automatically.*"

Step 4 — Trigger a Build

1. Click on "Run-Playwright-Tests"
2. Click "Build with Parameters" in the left sidebar
3. Show the three parameters:
   - **BASE_URL**: `http://frontend:8080` — "*This is the URL Playwright will test. It's the internal Docker network address of our frontend.*"

- **BROWSER_PROJECT**: `chromium` — *"We can test with Chromium, Firefox, or both. Chromium is faster and uses less memory."*
- **TEST_RETRIES**: `1` — *"If a test fails, Playwright will retry it once. This helps with flaky tests."*

4. Click **Build**

Step 5 — Watch the Build

1. Click on the build number (e.g., `#1`) in the Build History
2. Click **Console Output** in the left sidebar

> **Say**: *"Let's watch what happens in real time."*

3. Point out key moments in the log as they appear:
   - **"Installing npm dependencies"** — *"First it installs the Node.js packages."*
   - **"Browsers already installed at /ms-playwright"** — *"The browsers are pre-installed in the Docker image, so we skip the download."*
   - **"Health check against http://frontend:8080/api/healthz"** — *"Before running tests, it checks that the application backend is healthy. No point testing if the app is down."*
   - **"Backend is healthy!"** — *"We're good to go."*
   - **"Running Playwright tests"** — *"Now Playwright launches a headless Chromium browser and starts testing."*
   - **Lines like** `✓ [employee-chromium] › employee/submit-ticket-form.spec.ts` — *"Each test file runs and reports pass/fail."*
   - **"X passed"** at the end — *"All tests passed!"*

Step 6 — View the Report

1. Go back to the build page (click "Back to Run-Playwright-Tests" or the build number)
2. Click **Playwright Report** in the left sidebar

> **Say**: *"Jenkins publishes the Playwright HTML report. You can drill into each test, see what steps it took, and if any failed, see screenshots of what the browser showed at the moment of failure."*

3. Browse the report — click on a test to show its details

Step 7 — (Optional) Show the Shortcut: Running from Command Line

> **Say**: *"You can also run these exact same tests from your terminal, without Jenkins:"*

```
cd e2e
npm install
npx playwright test --project=employee-chromium --project=admin-chromium
npx playwright show-report    # Opens the HTML report in your browser
```

> **Say**: *"Jenkins adds value by providing a web dashboard, history, credential management, and team access. But the tests themselves are just Playwright — they can run anywhere Node.js is installed."*

Part 7 — How It All Connects

Here's the complete picture of how all the pieces fit together:

```
┌──────────────────────────────────────────┐
│              YOUR BROWSER                │
│                                          │
│   1. Open Jenkins (localhost:9090)       │
│   2. Redirect to Keycloak login          │
│   3. Log in as admin1                    │
│   4. Click "Build with Parameters"       │
```

```
│   5. Click "Build"                        │
└──────────────────┬──────────────────────────┘
                   │
┌──────────────────▼──────────────────────────┐
│              JENKINS                        │
│                                             │
│   6.  Clone Git repo (SCM step)             │
│   7.  Inject credentials (binding)          │
│   8.  Run ci-entrypoint.sh                  │
│       ├── npm install                       │
│       ├── Health check backend              │
│       └── npx playwright test               │
│               │                             │
└───────────────┼─────────────────────────────┘
                │
┌───────────────▼─────────────────────────────┐
│            PLAYWRIGHT                       │
│                                             │
│   9.  Launch headless Chromium              │
│   10. Navigate to HelpDesk app              │
│   11. Keycloak login (automated)            │
│   12. Run test scenarios:                   │
│       ├── Submit ticket ✓                   │
│       ├── Check dashboard ✓                 │
│       ├── Assign ticket ✓                   │
│       └── Filter/sort ✓                     │
│   13. Generate HTML report + screenshots    │
└───────────────┬─────────────────────────────┘
                │
┌───────────────▼─────────────────────────────┐
│            BACK IN JENKINS                  │
│                                             │
│   14. Collect Playwright report             │
│   15. Publish as "Playwright Report"        │
│   16. Archive test artifacts                │
│   17. Mark build SUCCESS or FAILURE         │
│                                             │
│   → You view the results in Jenkins UI      │
└─────────────────────────────────────────────┘
```

Project Files Quick Reference

File	Purpose
`jenkins/Dockerfile`	Custom Jenkins image with Node.js + Playwright browsers
`jenkins/plugins.txt`	Jenkins plugins to install
`jenkins/casc/jenkins.yaml`	JCasC: OIDC login, roles, credentials, job definition (local)
`jenkins/jobs/playwright-tests.groovy`	Reference-only copy of the Job DSL script (not used at runtime)
`e2e/playwright.config.ts`	Playwright config: browsers, projects, test settings
`e2e/tests/auth.setup.ts`	Logs in as employee + admin, saves session for other tests
`e2e/cleanup.ts`	Deletes test tickets before/after the test run
`e2e/pages/*.page.ts`	Page Objects: login, submit-ticket, admin-dashboard, navigation
`e2e/fixtures/test-data.ts`	Test data: ticket templates, user credentials, expected values
`e2e/scripts/ci-entrypoint.sh`	CI orchestrator: install → health check → export creds → run tests
`e2e/scripts/run-tests.sh`	Builds and runs the `npx playwright test` command
`docker-compose.yml`	Local dev environment (all 5 services including Jenkins)

File	Purpose
`helm/templates/jenkins-*.yaml`	OpenShift deployment templates (6 files)
`helm/values.yaml`	Helm values: Jenkins image, resources, Keycloak client, persistence

---

Part 8 — Going Deeper

In This Project

- **OpenShift & Helm Deep-Dive Guide** — Full Helm template walkthroughs for all components. The Jenkins section (3.5) covers every YAML field in detail.
- **Deployment Troubleshooting** — Resolved deployment issues, root causes, and diagnostic commands.

Official Documentation

- **Playwright**: https://playwright.dev/ — The official docs have excellent guides on writing tests, debugging, and configuration.
- **Jenkins Configuration as Code (JCasC)**: https://www.jenkins.io/projects/jcasc/ — How JCasC works, schema reference, and examples.
- **Jenkins Job DSL**: https://plugins.jenkins.io/job-dsl/ — The DSL plugin documentation and API reference for defining jobs in code.
- **OpenID Connect (OIDC) Basics**: https://openid.net/developers/how-connect-works/ — Understanding the protocol Jenkins uses to authenticate with Keycloak.

Part 9 — Three Alternative Ways to Run Tests with Jenkins

The current setup uses a **freestyle job** defined in JCasC that clones the Git repository and runs a shell script. That pattern works well for a demo, but real-world Jenkins installations typically use one or more of these three approaches instead.

> **Most common in practice**: approaches 9.1 (Jenkinsfile) and 9.3 (webhooks) are almost always used together — they are not really alternatives to each other. The team commits a `Jenkinsfile` to the repo, creates a Multibranch Pipeline job in Jenkins, and wires it to a webhook from their Git platform (Azure DevOps in an enterprise context). Every pull request triggers a test run automatically; merge is blocked until it passes. This is the standard in any actively developed project. Container agents (9.2) are also common, but usually layered on top: your `Jenkinsfile` declares `agent { docker { … } }` and you get both. The manual "click Build" model used in this project is typical only for demos, nightly scheduled jobs, or teams that have not yet automated their pipeline.

9.1 Jenkinsfile / Pipeline-as-Code

The idea: instead of defining the job in an external config file (like `jenkins/casc/jenkins.yaml`), you commit a file called `Jenkinsfile` directly to the root of the application repository. Jenkins reads it after cloning the code — the pipeline definition lives in the same repo as the code it tests.

```
YOUR REPOSITORY
├── frontend/
├── backend/
├── e2e/
├── Jenkinsfile          ← The entire pipeline definition lives HERE, in the repo
└── ...
```

The `Jenkinsfile` uses Jenkins' **Declarative Pipeline** syntax — a Groovy-based DSL. Here is what this project's job would look like as a `Jenkinsfile`:

```
pipeline {
    agent any

    parameters {
```

```
        string(name: 'BASE_URL', defaultValue: 'https://app.example.com')
        choice(name: 'BROWSER_PROJECT', choices: ['chromium', 'firefox', 'all'])
    }

    stages {
        stage('Install Dependencies') {
            steps {
                dir('e2e') { sh 'npm ci' }
            }
        }
        stage('Run Playwright Tests') {
            steps {
                withCredentials([
                    usernamePassword(credentialsId: 'employee-credentials',
                                     usernameVariable: 'EMPLOYEE_USERNAME',
                                     passwordVariable: 'EMPLOYEE_PASSWORD')
                ]) {
                    dir('e2e') { sh './scripts/ci-entrypoint.sh' }
                }
            }
        }
    }

    post {
        always {
            publishHTML(target: [reportDir: 'e2e/playwright-report', reportName: 'Playwright
Report'])
        }
    }
}
```

Why this matters over the current freestyle approach:

Aspect	Freestyle + JCasC (current)	Jenkinsfile Pipeline
Where is the pipeline defined?	`jenkins/casc/jenkins.yaml` (separate config repo or ConfigMap)	`Jenkinsfile` in the app repo itself
Code review	Pipeline changes bypass Git PR review	Pipeline changes go through the same PR process as application code
Visibility	Only Jenkins admins typically read the JCasC YAML	Every developer can read, understand, and propose changes
Automatic triggering	Must be wired up manually	Multibranch Pipeline scans all branches automatically

> **Multibranch Pipeline variant**: if you create a *Multibranch Pipeline* job type in Jenkins (instead of a regular pipeline job), Jenkins automatically scans every branch in the repository, finds the `Jenkinsfile` in each one, and creates a separate job per branch. Opening a pull request automatically provisions a dedicated test run for that branch — no manual configuration. This is the most common pattern in actively developed projects.

9.2 Docker / Kubernetes Container Agent

The idea: instead of running test commands directly on the Jenkins server (which requires Node.js, Playwright, and browsers to be pre-installed in the Jenkins image), each build step runs inside a **fresh, isolated container** that is created at build time and destroyed afterwards.

In the current project, Playwright and Chrome are baked into the Jenkins Docker image so they are available when the job runs. With container agents, the Jenkins image itself stays small and generic — the test container brings everything it needs.

There are always two pods in this model. This is the most important thing to understand:

```
┌──────────────────────────────────────────────────────────────────┐
│  POD 1 — Jenkins Controller  (permanent, always running)         │
│  ┌──────────────────────────────────────────────────────────────┐
│  │  • Jenkins web UI and dashboard                              │
│  │  • Job scheduler and build history                           │
│  │  • Credential store (passwords, PATs, secrets)               │
│  │  • No repo, no Node.js, no Playwright, no browsers           │
│  │                                                              │
│  │  Role: THE MANAGER — tells the worker what to do             │
│  └──────────────────────────────────────────────────────────────┘
│                    │
│   1. Build triggers
│   2. "I need a worker pod"
│   → Kubernetes plugin asks OpenShift to create Pod 2
└──────────────────────────────────────────────────────────────────┘
                     │
                     │ OpenShift provisions Pod 2
                     ▼
┌──────────────────────────────────────────────────────────────────┐
│  POD 2 — Jenkins Agent  (ephemeral, lives only for this build)   │
│  ┌──────────────────────────────────────────────────────────────┐
│  │  • Jenkins JNLP agent process  ←── connects back to Pod 1    │
│  │  • Git clone runs here         ← repo arrives at build time  │
│  │  • e2e/scripts/ci-entrypoint.sh  ← scripts are part of repo  │
│  │  • Node.js + Playwright + Chromium + Firefox                 │
│  │                                                              │
│  │  Role: THE WORKER — clones the repo, runs the scripts,       │
│  │         launches browsers, executes tests                    │
│  └──────────────────────────────────────────────────────────────┘
│       │
│   Pod 2 sends results back to Pod 1
│   OpenShift deletes Pod 2 when build ends — nothing left behind
└──────────────────────────────────────────────────────────────────┘
```

Where are the scripts?

They don't exist in Pod 2 until the build runs. Pod 2 starts as a blank container image (Node.js + Playwright + browsers, but no application code). The first thing the Jenkins agent process inside it does is clone the Git repository — the scripts (`ci-entrypoint.sh`, `run-tests.sh`, all the test files) materialise inside Pod 2's local workspace at that moment. Pod 1 never has a copy of the scripts. It just sends the `sh` command to Pod 2 over the JNLP connection, and Pod 2 executes it.

Compared to the current HelpDesk Pro project (where there is only one pod and Jenkins runs everything itself):

	Current project (1 pod)	**Container agent (2 pods)**
Pod 1 contains	Jenkins + Node.js + Playwright + Chrome	Jenkins only
Pod 2 contains	*(nothing — there is no Pod 2)*	JNLP agent + cloned repo + Node.js + Playwright + Chrome
Where tests run	Inside the Jenkins pod itself	Inside the agent pod
What happens after the build	Jenkins pod keeps running	Agent pod is deleted
Environment isolation	Previous run's state can linger	Always starts completely clean

In a Declarative Pipeline, the Docker variant looks like this:

```
pipeline {
    agent {
        docker {
```

```
            image 'mcr.microsoft.com/playwright:v1.42.0-jammy'  // Official Playwright image
            args '--shm-size=1g'                                  // Shared memory for Chrome
        }
    }
    stages {
        stage('Run Tests') {
            steps {
                dir('e2e') { sh 'npm ci && ./scripts/ci-entrypoint.sh' }
            }
        }
    }
}
```

Jenkins starts the `playwright:v1.42.0-jammy` container, runs the tests inside it, then discards it. The next build gets a clean, identical environment.

On OpenShift, the equivalent uses the Kubernetes plugin:

```
pipeline {
    agent {
        kubernetes {
            yaml '''
                apiVersion: v1
                kind: Pod
                spec:
                  containers:
                  - name: playwright
                    image: mcr.microsoft.com/playwright:v1.42.0-jammy
                    command: ["sleep", "infinity"]
                    resources:
                      requests:
                        memory: "1Gi"
            '''
        }
    }
    stages {
        stage('Run Tests') {
            steps {
                container('playwright') {
                    sh 'cd e2e && npm ci && ./scripts/ci-entrypoint.sh'
                }
            }
        }
    }
}
```

Jenkins asks OpenShift to provision a new Pod matching that spec, runs the tests inside it, and OpenShift deletes the Pod when done.

Key advantages:

- Test environment is always clean — if one run corrupts the Node modules cache or leaves browser state behind, the next run starts fresh
- Jenkins image stays lean — you can update the test container image independently of Jenkins
- Multiple different test suites can each declare their own container image with different dependencies

> **Why the current project does not use this**: baking browsers into the Jenkins image is simpler for a demo and removes the overhead of pod provisioning on each build. For a production team running many builds in parallel, container agents are the preferred approach because clean isolation and lean Jenkins images outweigh the startup cost.

9.3 Webhook-Triggered Multibranch Pipeline

The idea: instead of a developer clicking "Build with Parameters" in the Jenkins UI, Jenkins is triggered **automatically** whenever code is pushed to the repository. This transforms Jenkins from an on-demand tool into a fully automatic CI gate — tests run without any human action.

How webhooks work, end-to-end:

```
DEVELOPER            GIT REPOSITORY / AZURE DEVOPS        JENKINS
─────────            ─────────────────────────────        ───────

writes code
     │
     │  git push / opens PR
     ▼
branch updated ──────► ADO Service Hook fires ──────────────► Jenkins webhook endpoint
                       (HTTP POST with branch name,          receives notification
                        commit SHA, repo URL)                      │
                                                                   │  scans repo
                                                                   │  finds Jenkinsfile
                                                                   │  runs pipeline
                                                                   │
                       ADO PR shows ✓ or ✗  ◄──────────────── posts build status back
                       (commit status API)                   (pass / fail)
```

In Azure DevOps + OpenShift context, the wiring works like this:

1. **Azure DevOps** hosts the Git repository. You configure a **Service Hook** (ADO's term for a webhook) under Project Settings, pointed at Jenkins' URL at `/multibranch-webhook-trigger/`
2. **Jenkins** exposes that endpoint via the *Multibranch Scan Webhook Trigger* plugin
3. When ADO fires the event, Jenkins scans the branches that changed and runs only those pipelines
4. Jenkins posts the build result back to Azure DevOps via the ADO REST API or a Git *commit status* plugin — the PR shows a green check or red cross directly on the pull request page

Trigger modes compared:

Trigger	What Starts the Build	Typical Use Case
Manual (current project)	A person clicks "Build"	Demos, on-demand spot checks
Scheduled (cron)	A timer — e.g., every night at 2 AM	Nightly regression suites
Webhook on push	Every `git push` to any branch	Full CI — tests run on every commit
Webhook on PR	Opening or updating a Pull Request	PR gates — build must pass before merge

> **In an Azure DevOps shop**: the most common production pattern is the PR gate. A developer opens a pull request in ADO; ADO fires a webhook to Jenkins; Jenkins runs the tests against the branch; Jenkins posts the result back to the PR. Merge is blocked until the Jenkins build passes. This is almost certainly the pattern your client is running in production — understanding how webhooks wire ADO to Jenkins is what makes you sound like you have done this before.

Part 10 — Selenium: An Alternative UI Test Tool

> **Context**: this project uses Playwright for UI test automation. Selenium is the older, more widely deployed alternative. Many organisations built large Selenium suites long before Playwright existed and are not in a position to rewrite them.

10.1 What is Selenium?

Selenium is a **browser automation framework** that has been the industry standard for UI testing since around 2008 — over a decade before Playwright was released. The core idea is the same: write code that controls a real browser, clicks buttons, fills forms, and checks that the right things appear on screen. The fundamental difference is *how* that control happens at the protocol level.

Playwright talks to the browser directly. It uses the Chrome DevTools Protocol (CDP) — a WebSocket connection built into Chrome that gives Playwright direct access to the browser engine. No intermediary.

Selenium goes through a driver. Between your test code and the browser sits a separate executable called a *WebDriver*. For Chrome, this is `chromedriver`. For Firefox, it's `geckodriver`. Your code sends HTTP commands to the driver; the driver translates them into browser actions.

```
┌──────────────────────────────────────────────────────────────┐
│  SELENIUM ARCHITECTURE                                       │
│                                                              │
│  Test Code  ──HTTP──►  ChromeDriver  ──DevTools──►  Chrome   │
│  (Java/C#/             (separate       (browser              │
│   Python/JS)            process,        process)             │
│                         must version-                        │
│                         match Chrome)                        │
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
│  PLAYWRIGHT ARCHITECTURE                                     │
│                                                              │
│  Test Code  ──WebSocket──────────────────────────►  Chrome   │
│  (JS/TS/               (CDP — direct connection,    (bundled,  │
│   Python/Java)          no intermediary driver)      version-  │
│                                                      controlled) │
└──────────────────────────────────────────────────────────────┘
```

The driver-based model has one practical headache that Playwright avoids entirely: **the ChromeDriver version must exactly match the installed Chrome version**. When Chrome auto-updates on a CI server, tests silently break until someone manually updates `chromedriver`. Playwright sidesteps this by bundling its own browser builds — it always knows exactly which browser version it controls.

Language support is Selenium's biggest enterprise advantage:

Framework	Primary Languages	Common In
Selenium	Java, Python, C#, JavaScript, Ruby	.NET/Java enterprise shops, large legacy test suites
Playwright	JavaScript/TypeScript (primary); Python, Java, .NET also available	Modern web projects, Node.js teams

If a client has a large Java or C# team, a Selenium suite in the matching language is far more natural than migrating everyone to Playwright's JavaScript API. This is the most common reason enterprise organisations stick with Selenium.

10.2 How Selenium Works in Practice

A Selenium test in Java follows this pattern:

```java
// 1. Create a driver — this launches a real Chrome browser
WebDriver driver = new ChromeDriver();
driver.get("https://your-app.example.com");

// 2. Find elements and interact with them
WebElement usernameField = driver.findElement(By.id("username"));
usernameField.sendKeys("employee1");

WebElement passwordField = driver.findElement(By.id("password"));
passwordField.sendKeys("password123");

driver.findElement(By.id("kc-login")).click();

// 3. Assert the result
WebElement heading = driver.findElement(By.cssSelector("h1"));
```

```
assertEquals("My Tickets", heading.getText());

// 4. Always close the browser when done
driver.quit();
```

The interaction vocabulary (`findElement` → `sendKeys` / `click`) maps directly to what you already know from Playwright (`locator()` → `fill()` / `click()`). The structural difference is that Selenium's `findElement` **returns immediately** — it queries the current DOM snapshot at that instant. It does not wait for the element to appear or become interactive.

This is why explicit waits are mandatory in Selenium, and why their absence is the single biggest source of flaky Selenium tests:

```
// FRAGILE — element might not exist yet if the page is still loading
driver.findElement(By.id("ticket-list")).click();  // throws NoSuchElementException

// CORRECT — wait up to 10 seconds for the element to become visible
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement ticketList = wait.until(
    ExpectedConditions.visibilityOfElementLocated(By.id("ticket-list"))
);
ticketList.click();
```

> **Playwright does waits automatically.** When you call `page.locator('#ticket-list').click()` in Playwright, it polls the DOM for up to 30 seconds waiting for the element to be visible and stable before telling the browser to click. Forgetting `WebDriverWait` in Selenium — which many developers do — produces tests that pass on a fast developer machine and fail intermittently on a slower CI server. This is the most commonly cited frustration with Selenium in practice.

Selenium Grid scales this model to parallel multi-browser execution by separating test execution from browser execution:

```
┌──────────────────────────────────────────────────────────┐
│  SELENIUM GRID                                           │
│                                                          │
│  Test Code ──► Grid Hub ──► Chrome Node Pod  (Machine / Pod)  │
│                   │    ──► Firefox Node Pod (Machine / Pod)  │
│                   │    ──► Chrome Node Pod  (Machine / Pod)  │
│                                                          │
│  Hub: receives session requests, routes to an available node  │
│  Node: a machine or container with a browser + driver    │
└──────────────────────────────────────────────────────────┘
```

Your test connects to the Hub's URL using a `RemoteWebDriver` instead of a local driver — the Hub assigns the session to whichever Node has capacity:

```
URL gridUrl = new URL("http://selenium-hub:4444/wd/hub");
ChromeOptions options = new ChromeOptions();
WebDriver driver = new RemoteWebDriver(gridUrl, options);
// from here, the code is identical to local Selenium
```

This enables parallel cross-browser testing without installing multiple browsers on the same machine — each Node runs in its own container and can be scaled independently.

10.3 Integrating Selenium with Jenkins

From Jenkins' perspective, running Selenium tests looks almost identical to running Playwright tests. Jenkins executes a shell command. The JCasC credential injection, the credentials-binding plugin, the Git clone step — all of it carries over unchanged.

```
Jenkins Job (Selenium version)
│
├── SCM Step: clone repository                        (identical to Playwright)
│
├── Credentials Binding: inject test user passwords   (identical pattern)
│   └── EMPLOYEE_USERNAME, EMPLOYEE_PASSWORD, ADMIN_USERNAME, ADMIN_PASSWORD
│
├── Build Step: shell command
│   ├── Playwright:  cd e2e && npx playwright test
│   └── Selenium:    cd selenium-tests && mvn test -Dbase.url=$BASE_URL
│                    (or: pytest tests/ for Python Selenium)
│                    (or: dotnet test for C# Selenium (.NET / xUnit))
│
└── Post-Build: publish results
    ├── Playwright:  publishHTML plugin (Playwright HTML report)
    └── Selenium:    junit('target/surefire-reports/**/*.xml')
                     Jenkins JUnit plugin reads the standard JUnit XML that Maven/pytest produce
                     and renders pass/fail trends natively in the Jenkins UI
```

The key differences in practice:

Aspect	Playwright (current)	Selenium
Test command	`npx playwright test`	`mvn test` / `pytest` / `dotnet test`
Browser requirement	Pre-installed in Jenkins image (or container agent)	ChromeDriver + Chrome in image, or connect to external Grid
Result format	Playwright HTML report (htmlpublisher plugin)	JUnit XML (Jenkins JUnit plugin)
Parallel execution	Playwright project config (`workers`)	Selenium Grid node pool
Wait strategy	Automatic (built in)	Explicit `WebDriverWait` required in every test
Driver management	None — Playwright bundles its browsers	ChromeDriver version must match Chrome version

Everything from Part 3 of this guide — JCasC credentials, the credentials-binding plugin, the job DSL structure — applies equally to Selenium. You swap the test runner command and the result publisher. The Jenkins scaffolding is the same.

10.4 Three Selenium Architectures for Azure DevOps + OpenShift

> These three architectures are ordered from simplest to most scalable. All three run on OpenShift and integrate with Azure DevOps pipelines.

Architecture 1: Selenium on the Jenkins Agent

The simplest possible setup. ChromeDriver and Google Chrome are installed directly in the Jenkins container image — exactly as Playwright browsers are installed in this project. Tests run on the same pod as Jenkins, no external infrastructure needed.

```
OpenShift Cluster
┌──────────────────────────────────────────────────────────┐
│  Jenkins Pod                                             │
│  ┌──────────────────────────────────────────────────┐    │
│  │  Jenkins agent process                           │    │
│  │  + ChromeDriver (pre-installed)                  │    │
│  │  + Chrome headless (pre-installed)               │    │
│  │  + test framework (Maven / pytest / dotnet)      │    │
│  │                                                  │    │
│  │  mvn test  →  ChromeDriver  →  Chrome            │    │
```

```
│  │  (everything runs inside this one pod)            │  │
│  └──────────────────────────────────────────────────┘  │
└────────────────────────────────────────────────────────┘

          ▲
          │  Azure DevOps triggers via JenkinsQueueJob task
          │
Azure DevOps Pipeline ── developer pushes to ADO repo ──► pipeline starts
```

Azure DevOps integration:

Azure DevOps has a built-in **Jenkins service connection** under Project Settings → Service connections. Once configured with Jenkins' URL and credentials, an ADO pipeline can queue and wait on a Jenkins build:

```yaml
# azure-pipelines.yml
- task: JenkinsQueueJob@2
  inputs:
    serverEndpoint: 'Jenkins-OpenShift'      # the service connection name
    jobName: 'Run-Selenium-Tests'
    captureConsole: true                     # streams Jenkins log into ADO pipeline output
    captureJobParameters: true
```

This triggers the Jenkins job from within a step in the ADO pipeline, blocks until it finishes, and imports the console log — so the test output is visible in the ADO pipeline run without navigating to Jenkins.

When to use this architecture:

- Small test suite (under ~100 tests)
- Single browser (Chrome only is fine)
- Team without dedicated infrastructure budget
- Migrating an existing project — lowest-friction starting point

Key limitation: Chrome is memory-hungry (~500MB+). Tests run sequentially on one browser. A Chrome crash inside the pod can destabilise the Jenkins process, since they share the same pod.

Architecture 2: Selenium Grid with Docker Containers on OpenShift

The parallel testing setup. A dedicated Selenium Grid runs as separate pods alongside Jenkins. Jenkins connects to the Grid's Hub via a `RemoteWebDriver` URL — it hands off browser management entirely to the Grid. The Jenkins pod itself stays lean.

```
OpenShift Cluster
┌──────────────────────────────────────────────────────────────┐
│                                                              │
│   Jenkins Pod                  Selenium Grid                 │
│   ┌─────────────────┐          ┌──────────────────────────┐  │
│   │                 │          │  Hub Pod                 │  │
│   │  mvn test       │          │  routes sessions to nodes │  │
│   │  (RemoteDriver  │──── http ►│  http://selenium-hub:4444 │  │
│   │   URL to Hub)   │          └──────────┬───────────────┘  │
│   └─────────────────┘                     │                  │
│                                   ┌───────┴──────────────┐   │
│                                   │  Node Pods           │   │
│                                   │  ┌──────────┐ ┌──────┐│   │
│                                   │  │ Chrome   │ │Firefox│   │
│                                   │  │ Node x2  │ │Node x1│   │
│                                   │  └──────────┘ └──────┘│   │
│                                   └──────────────────────┘   │
└──────────────────────────────────────────────────────────────┘

          ▲
          │  Azure DevOps JenkinsQueueJob task + PublishTestResults task
```

Azure DevOps integration goes one step further in this architecture. After the test run, the JUnit XML results Jenkins produces can be uploaded directly to **Azure Test Plans**, giving the client test-trend visibility inside ADO without navigating to Jenkins:

```yaml
# azure-pipelines.yml
- task: JenkinsQueueJob@2
  inputs:
    serverEndpoint: 'Jenkins-OpenShift'
    jobName: 'Run-Selenium-Tests'
    captureConsole: true

- task: PublishTestResults@2
  inputs:
    testResultsFormat: 'JUnit'
    testResultsFiles: '**/TEST-*.xml'             # downloaded from Jenkins artifacts
    testRunTitle: 'Selenium Results - $(Build.BuildNumber)'
    mergeTestResults: true
```

This gives a pass/fail trend graph, flaky test tracking, and test history living in the Azure DevOps portal — not just in Jenkins.

Typical resource allocation for the Grid pods:

Pod	Replicas	Memory Request
Selenium Hub	1	256 MB
Chrome Node	2	1 GB each
Firefox Node	1	1 GB

When to use this architecture:

- Cross-browser testing required (Chrome + Firefox minimum)
- Parallel execution needed to keep CI run time under 10 minutes
- Team is comfortable with Docker and OpenShift
- Suite is in the 100–500 test range

Key limitation: Node pods run continuously even when no tests are executing, consuming cluster resources around the clock when idle. There is no auto-scaling in the classic Grid 3 model.

Architecture 3: Selenium Grid 4 Native Kubernetes on OpenShift (Elastic)

The cloud-native, elastic setup. Selenium Grid 4 introduced native Kubernetes support with a fully decomposed architecture. Instead of a monolithic Hub with always-on Node pods, Grid 4 splits into specialised components, and **browser pods are provisioned on demand** — they exist only for the duration of a test session, then the pod is automatically deleted.

```
OpenShift Cluster
┌──────────────────────────────────────────────────────────────────────────┐
│                                                                          │
│   Jenkins Pod                                                            │
│   ┌──────────────┐                                                       │
│   │  mvn test    │                                                       │
│   │  RemoteDriver│                                                       │
│   │  URL ────────────────────────────────────────────────────┐           │
│   └──────────────┘                                           │           │
│                                                              ▼           │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │  Selenium Grid 4 (deployed via Helm chart)                       │   │
│   │                                                                  │   │
│   │  ┌──────────┐   ┌──────────────────┐   ┌──────────────────┐     │   │
```

```
│   │  │ Router    │──►│ Distributor     │   │ Session Map       │     │  │
│   │  │ (entry    │   │ (assigns        │   │ (tracks which     │     │  │
│   │  │  point)   │   │  sessions to    │   │  test is on       │     │  │
│   │  └──────────┘   │  nodes; spawns  │   │  which node)      │     │  │
│   │                 │  pods on demand │   └──────────────────┘     │  │
│   │                 └────────────────┘                             │  │
│   │                          │                                     │  │
│   │                          │ creates pods when a session starts  │  │
│   │                ┌─────────┴──────────────────┐                  │  │
│   │                ▼                            ▼                  │  │
│   │          ┌──────────────┐              ┌──────────────┐        │  │
│   │          │ Chrome Pod   │              │ Firefox Pod  │        │  │
│   │          │ (spawned     │              │ (spawned     │        │  │
│   │          │  on-demand)  │              │  on-demand)  │        │  │
│   │          │ DELETED      │              │ DELETED      │        │  │
│   │          │ when done    │              │ when done    │        │  │
│   │          └──────────────┘              └──────────────┘        │  │
│   └──────────────────────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────────────┘

          ▲
          │
Azure DevOps ── full pipeline: build app → deploy to OpenShift → trigger tests → publish results
```

Azure DevOps plays a broader orchestration role in this architecture, because Architecture 3 typically forms part of a full CI/CD pipeline — not just a test trigger:

```
Azure DevOps Pipeline (azure-pipelines.yml)
│
├── Stage: Build
│   └── Build Docker images → push to Azure Container Registry
│
├── Stage: Deploy
│   └── helm upgrade --install (deploy updated app to OpenShift)
│
├── Stage: Test
│   └── JenkinsQueueJob@2 → Jenkins → Selenium Grid 4 → run full suite
│
└── Stage: Publish Results
    └── PublishTestResults@2 → JUnit XML → Azure Test Plans
        + ADO notification rules → Teams or Slack alerts on failure
```

Azure DevOps is the **outer CI/CD orchestrator** (build, deploy, report). Jenkins is the dedicated **test execution layer** — it manages the Grid lifecycle and handles the test runner logic. Neither tool is trying to do the other's job.

Deploying Selenium Grid 4 on OpenShift uses the official Selenium Helm chart:

```
helm repo add docker-selenium https://www.selenium.dev/docker-selenium
helm install selenium-grid docker-selenium/selenium-grid \
  --set isolateComponents=true \
  --set autoscaling.enabled=true \
  --set chromeNode.enabled=true \
  --set firefoxNode.enabled=true
```

The chart handles RBAC, ServiceAccounts, and the SecurityContextConstraints that OpenShift requires for browser pods to run headless as a non-root user.

When to use this architecture:

- Large test suite (500+ tests) requiring elastic scaling
- Multiple teams sharing the same Grid infrastructure

- OpenShift-native organisation already comfortable deploying via Helm
- Azure DevOps is the primary CI/CD platform; Jenkins is the specialised test runner
- Budget optimisation matters — elastic pods mean you only pay for compute when tests are actually running

---

Summary: all three Selenium architectures at a glance

Aspect	Arch 1: Jenkins Agent	Arch 2: Static Grid	Arch 3: Elastic Grid 4
Browser location	Same pod as Jenkins	Dedicated static Node pods	Ephemeral pods created per session
Parallelism	No — sequential only	Yes — fixed pool of nodes	Yes — elastic, scales with demand
Resource efficiency	Moderate	Low (nodes idle between builds)	High — pods exist only during tests
Infrastructure complexity	Low	Medium	High
Cross-browser	Chrome only (practical)	Chrome + Firefox	Chrome + Firefox + Edge
ADO integration	JenkinsQueueJob task	JenkinsQueueJob + JUnit publish to Test Plans	Full pipeline: build → deploy → test → report
Best for	Small teams / proof of concept	Medium teams, regular CI	Large organisations, enterprise scale

> **Most common in practice**: Architecture 2 (Static Selenium Grid) is the production standard for the majority of enterprise Selenium installations. Most teams that built a Selenium suite in the last decade are running a Hub + Chrome/Firefox Node setup — it is well-understood, well-documented, and has years of tooling around it. Architecture 1 (co-located on Jenkins) appears in smaller shops or older legacy setups where the suite never outgrew a single machine. Architecture 3 (Grid 4 elastic) is where the industry is clearly heading — particularly on cloud-native platforms like OpenShift — but it remains the minority in production today. If your client has a mature Selenium suite on OpenShift running with Azure DevOps, the safe assumption is a static Grid (Architecture 2) triggered by ADO webhooks via a Jenkinsfile pipeline. Knowing the path forward to Grid 4 — and being able to articulate why elastic pod provisioning is better — is what separates a junior answer from a senior one.